seeStack — Final Project Report

> A focused monitoring platform for developers. seeStack captures application errors from user-built apps and continuously monitors the availability of websites, presenting both in a single web dashboard.

---

1. Project Overview

seeStack is a developer monitoring tool built around two core responsibilities:

1. **Error Monitoring.** Applications report runtime exceptions to seeStack through a lightweight SDK. Every event is stored, and repeated occurrences of the same bug are grouped together by a stable fingerprint so noise does not drown the signal.
2. **Website Monitoring.** Users register URLs they care about. seeStack performs periodic HTTP checks, records whether each target was reachable, and surfaces current status, latency, and uptime history.

The system is self-contained: a user registers with an email and password, creates a project, drops the SDK into their codebase with the project's ingest key, and immediately sees live data in the dashboard.

---

2. Problem Statement

Running software in production raises two recurring questions:

- **"Is my application throwing errors I don't know about?"** Without a centralised tracker, exceptions live in scattered server logs and are discovered only when a user complains.
- **"Is my website still up?"** Manual HTTP checks do not scale and usually fail to detect outages before customers do.

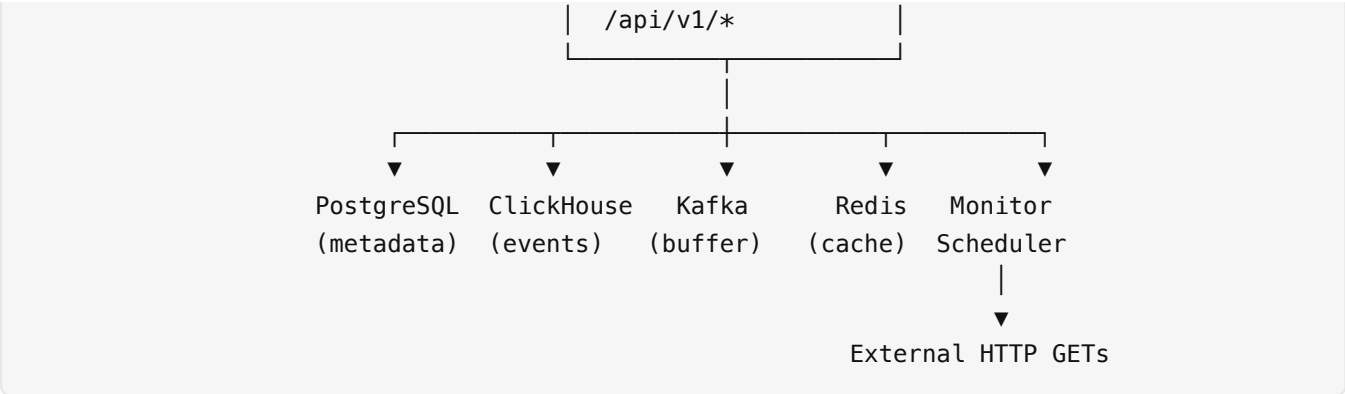
Commercial observability platforms answer these questions but are often overbuilt, tied to external identity providers, and priced for large organisations. seeStack answers the same two questions with a small, self-hosted system that a developer can run end-to-end on one machine.

---

3. System Architecture

seeStack follows a clean, pragmatic architecture:

```
┌──────────────┐                    ┌──────────────────────┐
│  Application │──SDK──► HTTPS ──►  │   Spring Boot API    │
└──────────────┘   ingest key        │  /ingest/v1/errors   │
```



3.1 Backend

Spring Boot 3 on Java 17. Modules:

- `auth` — email + password registration and login; issues an HS256 JSON Web Token used by the dashboard.
- `teams` — users and projects. A user owns one or more projects.
- `errors` — ingest endpoint, grouping service, and dashboard API for listing and updating error issues.
- `monitors` — CRUD for monitored URLs and the background scheduler that executes checks.
- `shared/security` — JWT and project-ingest-key filters.
- `ingestion` — Kafka producers and consumers plus the ClickHouse writers.

3.2 Frontend

React + Vite + TypeScript single-page application. Feature slices mirror the backend modules:

- `auth` — Login and Register pages.
- `overview` — Summary dashboard.
- `projects` — Project list, creation, ingest-key reveal.
- `errors` — Issue list page and a dedicated error detail page.
- `monitors` — Monitor grid with status cards and per-monitor detail.
- `sdk-setup` — Copy-ready install, initialise, and capture snippets for JavaScript, Java, and Python.

3.3 Storage

- **PostgreSQL 16** — relational metadata: `users`, `projects`, `api_keys`, `monitor_configs`, `error_groups`.
- **ClickHouse 24** — high-volume time-series events: `seestack.errors` with a 30-day TTL, and `seestack.monitor_checks`.

3.4 Messaging and scheduling

- **Apache Kafka (KRaft mode)** buffers incoming events so the ingest API returns immediately and writes to ClickHouse proceed asynchronously.
- **Redis** provides cache storage.

- A **Spring `@Scheduled` scheduler** wakes every minute, picks the monitors whose interval has elapsed, performs a real HTTP GET, and publishes the result to Kafka for storage in ClickHouse.

3.5 Runtime topology

All infrastructure runs as Docker containers (`docker compose`). The backend is a runnable Spring Boot jar and the frontend is a Vite dev server in development / a static bundle in production.

---

4. Core Features

4.1 Error Monitoring

- Public ingest endpoint authenticated by a project-scoped ingest key.
- Stable SHA-256 fingerprint over the exception class and the top call-site frames; repeated occurrences land in a single grouped issue with an incrementing occurrence count.
- Filtering by status, severity level, and environment.
- Dedicated full-page error detail view with stack trace, occurrence count, timestamps, and project association.

4.2 Website Monitoring

- Create / list / delete monitors through the dashboard.
- Configurable check interval per monitor.
- Automatic classification: a check counts as *up* when the response code is `2xx` or `3xx` **and** the response arrives within 30 s; otherwise *down*.
- Per-monitor history stored in ClickHouse; the list endpoint enriches each row with current status, last response time, and uptime percentage.

4.3 Projects Management

- A user owns one or more projects. Errors and monitors belong to a project.
- Creating a project generates a default ingest key, shown once in the UI right after creation for copy-and-paste into the SDK.
- Each project carries a safe key preview (`ask_live_…`) visible afterwards for identification.

4.4 SDK Integration

- Zero-dependency client SDKs in JavaScript, Java, and Python. Each exposes a single `SeeStack` class with a `captureException` method.
- All three SDKs target the same HTTP endpoint using the `X-SeeStack-Key` header. Nothing else is required.
- A built-in **SDK Setup** page in the dashboard renders install, initialise, and capture snippets pre-filled with the selected project's ingest key.

4.5 Authentication

- `POST /api/auth/register` — name, email, password. Passwords are hashed with BCrypt.
- `POST /api/auth/login` — returns a signed HS256 JWT.
- A security filter chain validates the JWT for `/api/v1/**` requests and the ingest key for `/ingest/v1/**` requests.

---

5. Technologies Used

Layer	Technology
Backend language	Java 17
Backend framework	Spring Boot 3.4
Password hashing	BCrypt
Dashboard JWT	HS256 (in-house issuance and validation)
Relational storage	PostgreSQL 16
Time-series storage	ClickHouse 24
Messaging	Apache Kafka 3.8 (KRaft mode)
Cache	Redis 7
Scheduled HTTP checks	JDK `java.net.http.HttpClient`
Frontend	React · Vite · TypeScript
SDK languages	JavaScript · Java · Python
Runtime	Docker & Docker Compose
Build	Gradle (backend) · pnpm (frontend)

---

6. System Workflow

6.1 Error ingestion

```
SDK.captureException(error)
        │
        ▼
POST /ingest/v1/errors     (header: X-SeeStack-Key)
        │
        ▼
ApiKeyAuthFilter resolves the project
        │
```

```
        ▼
ErrorFingerprintService
        = SHA-256(exceptionClass + top-5 meaningful stack frames)
        │
        ▼
Kafka topic  seestack.errors  (returns 202 Accepted immediately)
        │
        ▼
Consumer writes:
        ├─▶ ClickHouse  seestack.errors           (raw event)
        └─▶ PostgreSQL  error_groups              (grouped issue)
```

Repeat events with the same fingerprint increment the `occurrences` counter on the existing `error_groups` row.

6.2 Website monitoring

```
POST /api/v1/monitors   (Bearer JWT)
        │
        ▼
monitor_configs row created in PostgreSQL
        ▲
        │
MonitorScheduler @ fixedRate 60 s
        │
        ▼
HttpClient.send(GET url, timeout = 30 s)
        │
up  ⇐  status in [200, 400) AND elapsed < 30 s  ⇒  down
        │
        ▼
Kafka topic  seestack.monitor-checks
        │
        ▼
ClickHouse  seestack.monitor_checks
        ▲
        │
GET /api/v1/monitors          (enriched with status, uptime %, last response time)
GET /api/v1/monitors/{id}/checks   (historical check series)
```

6.3 Dashboard visualisation

- **Overview** aggregates counts (total errors, unresolved errors, monitors up, monitors down) and shows a recent-errors list plus a monitor status panel.
- **Projects** lists the user's projects and lets them create new ones.
- **Errors** lists grouped issues with status / severity / environment filters. A row click opens the dedicated detail page at `/errors/:fingerprint`.
- **Monitors** shows a card per monitor with live status and uptime.

- **SDK Setup** presents installation, initialisation, and capture snippets in JavaScript, Java, and Python — all pre-filled with the selected project's ingest key.

---

7. Team Responsibilities

Hi Dr., I hope you are doing well.

The responsibilities were distributed across the team as follows:

- **Abdualziz Alosaimi & Salem Omran** — Backend development and system architecture.
- **Mohammed Taleb & Mohammed bin Eid** — Frontend development and dashboard implementation.
- **Omar Alallaf & Faisal Alghanim** — Testing and documentation.

---

8. Achievements

- Delivered a complete monitoring platform with a focused feature set: error tracking and website uptime, exposed through a single dashboard.
- Designed and implemented a Kafka-buffered ingest pipeline that accepts error events, groups repeats by a stable fingerprint, and stores raw events in a columnar database suited to time-series queries.
- Built a scheduled HTTP-check engine that classifies each check deterministically and records full history for uptime reporting.
- Produced three first-party SDKs (JavaScript, Java, Python), each a zero-dependency single-file client, with a dashboard page that generates ready-to-paste integration code per project.
- Implemented a clean, self-contained authentication layer using BCrypt-hashed passwords and HS256 JSON Web Tokens.
- Packaged the whole stack so it can be started locally with a single command, with a seeded demo account and seeded demo data for immediate exploration.

---

9. Conclusion

seeStack answers two concrete engineering questions: *are my applications throwing errors?* and *are my websites up?* It does so with a coherent, self-contained architecture — a Spring Boot backend, a React dashboard, PostgreSQL for metadata, ClickHouse for events, Kafka for buffered ingest, and internal JWT authentication.

For the team, the project delivered practical experience with production-grade building blocks: asynchronous ingest, polyglot persistence, scheduled work, stable error grouping, and a modern

React frontend connected through a typed API client. For a developer adopting it, seeStack provides immediate, usable visibility into application errors and website availability in one place.

---

10. Runtime Validation

The system was exercised end-to-end against the running stack. The following results were captured live.

10.1 Infrastructure and backend

- `docker compose up` brings up four infrastructure containers: PostgreSQL, ClickHouse, Kafka, and Redis.
- The backend reports `{"status":"UP"}` at `GET /actuator/health` after startup.
- The schema is applied automatically: six PostgreSQL tables — `users`, `projects`, `api_keys`, `monitor_configs`, `error_groups`, `flyway_schema_history` — and two ClickHouse tables — `seestack.errors` and `seestack.monitor_checks`.

10.2 Authentication and projects

- `POST /api/auth/register` creates a user and returns a JSON Web Token along with a default project and its ingest key.
- `POST /api/auth/login` authenticates an existing user and returns a fresh token.
- `POST /api/v1/projects` creates a project and returns the ingest key **once** in the response. Subsequent `GET /api/v1/projects` responses include only the safe key prefix.

10.3 Error monitoring

- Running the JavaScript SDK example against the running backend sends three events and each is acknowledged with HTTP `202`.
- The `error_groups` table holds the expected number of grouped issues: repeated events with the same fingerprint share a single row with an incremented `occurrences` counter, and distinct events appear as separate rows.
- The ClickHouse `seestack.errors` table holds one raw row per event.
- The dashboard `/errors` page renders the grouped issues, and clicking a row opens `/errors/:fingerprint` — a full detail page with exception class, message, stack trace, occurrence count, first/last seen timestamps, and project association.
- The Python and Java SDK examples were also executed against the running backend and produced `202` responses; their events appear in the dashboard alongside the JavaScript events.

10.4 Website monitoring

- Two monitors are created by the seed flow: a healthy URL (`https://example.com`) and an unreachable URL (`http://127.0.0.1:1/nope`).

- Within one minute the scheduler records both checks in `seestack.monitor_checks`: the healthy URL produces a row with `status = 1` and a 200 status code; the unreachable URL produces a row with `status = 0` and a status code of 0.
- The dashboard `/monitors` page displays the two monitors with their classifications, last response time, and uptime percentage.

10.5 Dashboard validation

Navigating the dashboard via a browser with the demo account:

- The sidebar exposes exactly: **Overview · Projects · Errors · Monitors · SDK Setup**.
- **Overview** shows seeded counts — total errors, unresolved errors, monitors up, monitors down — along with a recent-errors list and a monitor status panel.
- **Projects** lists the seeded demo project with its safe key prefix visible.
- **Errors** lists the seeded grouped issues.
- **Errors detail** renders the full per-issue page with all metadata.
- **Monitors** renders both seeded monitors.
- **SDK Setup** shows install, initialisation, and capture snippets for JavaScript, Java, and Python, each pre-filled with the selected project's ingest key and a copy button.

10.6 Summary matrix

Check	Result
Infrastructure containers healthy	✅
Backend `/actuator/health` returns UP	✅
Database schema applied automatically	✅
Register and login succeed with JWT issuance	✅
Project creation returns a one-time ingest key	✅
SDK events acknowledged with HTTP 202	✅
Grouped issues stored in PostgreSQL `error_groups`	✅
Raw events stored in ClickHouse `seestack.errors`	✅
Scheduler writes UP and DOWN checks to ClickHouse	✅
Dashboard renders Overview, Projects, Errors, Monitors, SDK Setup	✅
Error detail page renders per-issue data	✅
Sidebar exposes exactly the five intended sections	✅